CONNECTED COMPONENTS

Use when: counting/grouping separate 'islands' in a graph.

```
vector<bool> visited(n, false); int numComponents = 0;
for (int i=0;i<n;i++) {
    if (!visited[i]) { numComponents++; dfs(i); }  // reuse dfs from Page 20
}
```

CYCLE DETECTION

Undirected graph: a visited neighbor that isn't the parent means a cycle.

```
bool hasCycleUndirected(int u, int parent, vector<bool>& vis) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) { if (hasCycleUndirected(v, u, vis)) return true; }
        else if (v != parent) return true;
    }
    return false;
}
```

Directed graph: track the current recursion path — a back-edge into it means a cycle.

```
vector<int> state;  // 0=unvisited, 1=in progress, 2=done
bool hasCycleDirected(int u) {
    state[u] = 1;
    for (int v : adj[u]) {
        if (state[v] == 1) return true;  // back-edge to in-progress node
        if (state[v] == 0 && hasCycleDirected(v)) return true;
    }
    state[u] = 2; return false;
}
```

BIPARTITE CHECK

Use when: checking if a graph's nodes can be 2-colored with no adjacent same color.

```
vector<int> color(n, -1);
bool isBipartite(int start) {
    queue<int> q; q.push(start); color[start] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (color[v] == -1) { color[v] = 1 - color[u]; q.push(v); }
            else if (color[v] == color[u]) return false;
        }
    }
    return true;
}
```

FLOOD FILL

Use when: filling/marking a connected region — grid version of DFS/BFS.

```
void floodFill(int r, int c, vector<vector<int>>& grid, int target, int newVal) {
    if (r<0||r>=grid.size()||c<0||c>=grid[0].size()||grid[r][c]!=target) return;
    grid[r][c] = newVal;
    floodFill(r+1,c,grid,target,newVal); floodFill(r-1,c,grid,target,newVal);
    floodFill(r,c+1,grid,target,newVal); floodFill(r,c-1,grid,target,newVal);
}
```

===== DIRECTED GRAPH INDEGREE =====

```
vector<int> indegree(n, 0);
for (int u=0; u<n; u++)
    for (int v : adj[u]) indegree[v]++;
```